

A Parallel Algorithm for Computing Fourier Transforms on the Star Graph

Paraskevi Fragopoulou, *Student Member, IEEE*, and Selim G. Akl, *Senior Member, IEEE*

Abstract—The n -star graph, denoted by S_n , is one of the graph networks that have been recently proposed as attractive alternatives to the n -cube topology for interconnecting processors in parallel computers. In this paper, we present a parallel algorithm for the computation of the Fourier transform on the star graph. The algorithm requires $O(n^2)$ multiply-add steps for an input sequence of $n!$ elements, and is hence cost-optimal with respect to the sequential algorithm on which it is based. To the best of our knowledge, this is the first algorithm, and the only one to date, for the computation of the Fourier transform on the star graph.

Index Terms—Cost optimality, fast Fourier transform, interconnection network, parallel algorithm, star graph

I. INTRODUCTION

THE STAR GRAPH was proposed in [1] as an attractive alternative to the n -cube (or hypercube) topology for interconnecting processors in parallel computers. It offers a network with fewer interconnection edges and smaller communication delays compared to the hypercube. Because of its rich symmetry, the star graph has various decomposition schemes.

In this paper, we present a parallel algorithm for the computation of the Fourier transform on the star graph. To the best of our knowledge, this is the first such algorithm (and the only one to date) to be proposed. The algorithm is based on a method for the computation of the Fourier transform initially presented by Cooley and Tukey in 1964 [6]. We use a specific decomposition of the star graph that combines two known decomposition schemes. These are the decomposition of S_n into n subgraphs S_{n-1} , and into $(n-2)!$ disjoint cycles of length $(n-1)n$ each. For an input sequence of $n!$ elements, the algorithm requires $O(n^2)$ multiply-add steps, and is cost-optimal with respect to the sequential algorithm on which it is based.

The paper is organized as follows. In Section II, we give a description of the star graph as well as some basic properties. A scheme for decomposing the star graph into disjoint cycles is described in Section III, and in Section IV an ordering for the vertices of the star graph is given. In Section V, we present the Cooley–Tukey method for the computation of the Fourier transform. Our parallel algorithm appears in Section VI, and in Section VII we show how the Fourier transform can be computed on a linear array of processors, as required

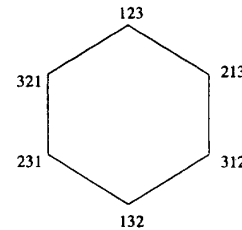


Fig. 1. The S_3 : A hexagon.

in various substeps of the main algorithm. Finally, in the last section, we analyze the algorithm and compare it with other parallel algorithms.

II. THE STAR GRAPH

The n -star graph, denoted by S_n , has $n!$ vertices corresponding to the $n!$ permutations of n distinct symbols. A vertex corresponding to permutation $a_1 a_2 \cdots a_{i-1} a_i a_{i+1} \cdots a_n$ is connected to those vertices corresponding to permutations $a_i a_2 \cdots a_{i-1} a_1 a_{i+1} \cdots a_n$, for $2 \leq i \leq n$, (i.e., those permutations that result from interchanging the first symbol in the permutation $a_1 a_2 \cdots a_{i-1} a_i a_{i+1} \cdots a_n$ with any of the remaining $n-1$ symbols). The edge connecting the vertex associated with the original permutation to that vertex associated with the permutation resulting from the interchange between the first and the i th symbols is called the i th connection. In this way, every vertex is an endpoint of $n-1$ edges corresponding to the $n-1$ symbols that can be interchanged with the symbol in the first position of the associated permutation, as shown in Fig. 1 for the three symbols 1, 2, and 3 [1]. Without loss of generality, we henceforth use the set of symbols $\{1, 2, \dots, n\}$ to label the vertices of S_n .

As shown in [1], [2], S_n enjoys a number of properties desirable in interconnection networks. These include regularity, vertex and edge symmetry, maximal fault tolerance, and strong resilience. Because of its rich symmetry, the graph is easily extensible, can be decomposed in various ways, and allows for simple routing algorithms. The graph was shown to be Hamiltonian in [4], [9], and an efficient sorting algorithm was developed for it in [8].

In addition, S_n is superior to the n -cube [1] with respect to three key properties: total number of vertices, degree (number of edges at each vertex), and diameter (maximum distance between any two vertices). For any $n > 3$, S_n has more vertices than the n -cube ($n!$ vertices versus 2^n), and the degree

Manuscript received March 3, 1992; revised September 14, 1992. This work was supported by the Telecommunications Research Institute of Ontario and by the Natural Sciences and Engineering Research Council of Canada.

The authors are with the Department of Computing and Information Science, Queen's University, Kingston, ON K7L 3N6, Canada.

IEEE Log Number 9216781.

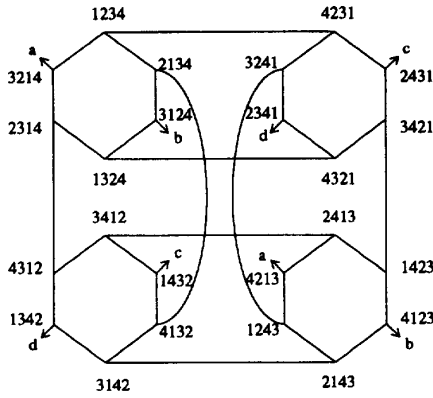


Fig. 2. The S_4 : Four interconnected S_3 .

and diameter of S_n are sublogarithmic in the number of its vertices ($n-1$ and $\lfloor \frac{3(n-1)}{2} \rfloor$, respectively) compared to those of the n -cube, which are both equal to n , i.e., logarithmic in the number of its vertices.

It is also known that S_n can be decomposed into n subgraphs S_{n-1} by fixing each different symbol in one particular position 2 to n [1]. If we fix a symbol in the last position, we observe that there are $(n-1)!$ permutations that constitute an S_{n-1} . For example, if the symbol in the last position is held fixed with any symbol, then we get $(n-1)!$ permutations (i.e., an S_{n-1}) for every one of the n symbols. Thus, the vertices of the S_n can be partitioned into n groups, each containing $(n-1)!$ vertices and each being isomorphic to S_{n-1} as shown in Fig. 2.

Let m_1 and m_2 be two distinct symbols from $\{1, 2, \dots, n\}$. Further, let $*$ represent any permutation of the $n-2$ symbols $\{1, 2, \dots, n\} - \{m_1, m_2\}$. In what follows, we use the notation $m_1 * m_2$ to represent a permutation of $\{1, 2, \dots, n\}$. For example, if $n = 9$, m_1 is 5, m_2 is 8, and $*$ is 7416932, then $m_1 * m_2$ is 574169328.

III. A DECOMPOSITION SCHEME FOR THE STAR GRAPH

Recently, a new decomposition scheme was proposed in [13], which partitions S_n into $(n-2)!$ disjoint cycles of length $(n-1)n$ each. Each cycle can be defined by its initial permutation. Initial permutations in S_n are of the form $P = 1 * n$. If we represent by d_i the position of symbol i in P , where $d_1 = 1$, then starting at P and repeatedly visiting connections $d_2, d_3, \dots, d_{n-1}, d_n, d_2, \dots$, we get a cycle of length $(n-1)n$. The way a cycle is created is described below.

Starting at P and visiting connections $d_2, d_3, \dots, d_{n-1}$, we get a path $1 * n, 2 * n, \dots, (n-1) * n$ of length $n-1$, in which all permutations end with symbol n . If we now visit connection d_n , we get the permutation $n * (n-1)$, in which symbol i is in position $d_{i \bmod n+1}$. Starting from $n * (n-1)$, we visit once again connections $d_2, d_3, \dots, d_{n-1}$, and we get the path $n * (n-1), 1 * (n-1), \dots, (n-2) * (n-1)$ of $n-1$ permutations, all of which end with the symbol $n-1$. In general, the j th, $1 \leq j \leq n$, time we visit connections $d_2, d_3, \dots, d_{n-1}$, we get the path $(n-j+2) * (n-j+1), (n-j+3) * (n-j+1), \dots, n * (n-j+1), 1 * (n-j+1), \dots, (n-j) * (n-j+1)$

of length $n-1$, in which all permutations end with symbol $n-j+1$. If we call the j th group of $n-1$ permutations path j , the following observations can be made.

- 1) The first permutation of path j is obtained from the last permutation of path $j-1$ through connection d_n .
- 2) In the first permutation of path j (obtained the $(j-1)$ st time we visit connection d_n), symbol i is in position $d_{(i+j-1) \bmod n+1}$. The n th time we visit connection d_n , we get a permutation in which symbol i is in position $d_{(i+n-1) \bmod n+1} = d_i$ (since $1 \leq i \leq n$); in other words, we get P . Therefore, we get a cycle.
- 3) We have n paths of $n-1$ permutations each. The permutations in each path are distinct, and each permutation in path i is different from each permutation in path j ($i \neq j$), because permutations in different paths end with different symbols. As a consequence, a cycle contains $(n-1)n$ distinct permutations.
- 4) Because each cycle is generated by an initial permutation of the form $1 * n$, there are in total $(n-2)!$ cycles, one for each permutation of the symbols $2, 3, \dots, n-1$.
- 5) All the cycles of S_n are disjoint. For a formal proof of this property, see [13].

For example, the two cycles contained in S_4 consist of the following permutations.

cycle 1:	path 1:	1234 $\xrightarrow{d_2=2}$ 2134 $\xrightarrow{d_3=3}$ 3124 $\xrightarrow{d_4=4}$
	path 2:	4123 $\xrightarrow{d_2=2}$ 1423 $\xrightarrow{d_3=3}$ 2413 $\xrightarrow{d_4=4}$
	path 3:	3412 $\xrightarrow{d_2=2}$ 4312 $\xrightarrow{d_3=3}$ 1342 $\xrightarrow{d_4=4}$
	path 4:	2341 $\xrightarrow{d_2=2}$ 3241 $\xrightarrow{d_3=3}$ 4231 $\xrightarrow{d_4=4}$ 1234
cycle 2:	path 1:	1324 $\xrightarrow{d_2=3}$ 2314 $\xrightarrow{d_3=2}$ 3214 $\xrightarrow{d_4=4}$
	path 2:	4213 $\xrightarrow{d_2=3}$ 1243 $\xrightarrow{d_3=2}$ 2143 $\xrightarrow{d_4=4}$
	path 3:	3142 $\xrightarrow{d_2=3}$ 4132 $\xrightarrow{d_3=2}$ 1432 $\xrightarrow{d_4=4}$
	path 4:	2431 $\xrightarrow{d_2=3}$ 3421 $\xrightarrow{d_3=2}$ 4321 $\xrightarrow{d_4=4}$ 1324

The decomposition of S_n into disjoint cycles can be combined with its decomposition into substars. It is well known that each S_n can be decomposed into $n!/k!$ substars S_k ($k < n$) [1]. We have just demonstrated how to decompose S_k into $(k-2)!$ cycles each of length $(k-1)k$. As a consequence, S_n can be decomposed into $n!/(k-1)k$ disjoint cycles of length $(k-1)k$ each.

IV. ORDERING OF THE VERTICES

In this section, we give an ordering for the vertices of S_n , and a function that maps this ordering to the positive integers. As shown in Section VI, this ordering allows input to, and output from, the vertices of the star graph in a manner appropriate for the computation of the Fourier transform.

The ordering $<$ on the vertices of S_n can be defined as follows: We say that $a_1 a_2 \dots a_i \dots a_n < b_1 b_2 \dots b_i \dots b_n$ if there exists an $i, 1 \leq i \leq n$, such that $a_j = b_j$ for $j > i$, and $a_i > b_i$. In order to provide some intuition, we relate $<$ to the more common lexicographic ordering. Assume that the

TABLE I
PERMUTATIONS OF THE FOUR SYMBOLS {1, 2, 3, 4} ORDERED
ACCORDING TO THE ORDERING DEFINED IN SECTION IV

1234	(1)	1243	(7)	1342	(13)	2341	(19)
2134	(2)	2143	(8)	3142	(14)	3241	(20)
1324	(3)	1423	(9)	1432	(15)	2431	(21)
3124	(4)	4123	(10)	4132	(16)	4231	(22)
2314	(5)	2413	(11)	3412	(17)	3421	(23)
3214	(6)	4213	(12)	4312	(18)	4321	(24)

vertices have been listed according to $\prec$. If this list were to be traversed in reverse order, while reading the symbols of each permutation from right to left, lexicographic order would be obtained. For example, for $n = 4$, the permutations of {1, 2, 3, 4} ordered according to $\prec$ are given in Table I (in column major order).

We now describe a function that maps the permutations, ordered according to $\prec$, into the first $n!$ positive integers. If $a_1 a_2 \dots a_i \dots a_n$ is a permutation, then for each symbol a_i of the permutation, we define π_i to be as follows:

$$\pi_i = \left| a_i - i - \sum_{j=i+1}^n [a_i > a_j] \right| (i-1)! \text{ for } 2 \leq i \leq n,$$

where $[a_i > a_j]$ equals 1 when $a_i > a_j$, and equals 0 otherwise. Then the number of permutation $a_1 a_2 \dots a_i \dots a_n$ is as follows:

$$\Pi(a_1 a_2 \dots a_i \dots a_n) = 1 + (\pi_2 + \pi_3 + \dots + \pi_n).$$

This mapping function resembles the one appearing in [3] for the ordering of permutations in lexicographic order.

V. THE COOLEY-TUKEY FAST FOURIER TRANSFORM

Given a sequence of numbers $\{x(1), x(2), \dots, x(N)\}$, its Fourier transform (FT) is the sequence resulting from evaluating the following expression:

$$X(j) = \sum_{k=1}^N x(k) w_N^{(j-1)(k-1)} \text{ for } j = 1, 2, \dots, N,$$

where w_N is a primitive N th root of unity, i.e., $w_N = e^{2\pi i/N}$ with $i = \sqrt{-1}$.

Suppose that N , the number of input elements, is composite, and factors as $N = r_1 \cdot r_2$. If we arrange the N elements into an $r_1 \times r_2$, 2-D grid using row major order, the Fourier transform can be written as follows:

$$\begin{aligned} X(j_1, j_0) &= \sum_{k_0=1}^{r_2} \sum_{k_1=1}^{r_1} x(k_1, k_0) \\ &\quad \cdot w_N^{[(j_1-1)r_1+j_0-1][(k_1-1)r_2+k_0-1]} \\ &= \sum_{k_0=1}^{r_2} \sum_{k_1=1}^{r_1} x(k_1, k_0) \\ &\quad \cdot w_N^{(j_1-1)(k_1-1)r_1 r_2} w_N^{(j_1-1)(k_0-1)r_1} \\ &\quad \cdot w_N^{(j_0-1)(k_1-1)r_2} w_N^{(j_0-1)(k_0-1)}, \end{aligned}$$

where $x(k_1, k_0)$ refers to the element in row k_1 and column k_0 of the grid,

$$\begin{aligned} j &= (j_1 - 1)r_1 + j_0, & j_0, k_1 &= 1, 2, \dots, r_1, \\ k &= (k_1 - 1)r_2 + k_0, & j_1, k_0 &= 1, 2, \dots, r_2. \end{aligned}$$

Since $w_N^{(j_1-1)(k_1-1)r_1 r_2} = w_N^{(j_1-1)(k_1-1)^N} = (e^{2\pi i})^{(j_1-1)(k_1-1)} = 1$, we have the following:

$$\begin{aligned} X(j_1, j_0) &= \sum_{k_0=1}^{r_2} w_N^{(j_1-1)(k_0-1)r_1} [w_N^{(j_0-1)(k_0-1)} \\ &\quad \cdot \sum_{k_1=1}^{r_1} x(k_1, k_0) w_N^{(j_0-1)(k_1-1)r_2}]. \end{aligned}$$

According to this decomposition method, the Fourier transform can be computed in three steps [6].

- 1) $X'(j_0, k_0) = \sum_{k_1=1}^{r_1} x(k_1, k_0) w_{r_1}^{(j_0-1)(k_1-1)}$
(Fourier transform of each column of the grid);
- 2) $X''(j_0, k_0) = w_N^{(j_0-1)(k_0-1)} X'(j_0, k_0)$
(local multiplications); and
- 3) $X(j_1, j_0) = \sum_{k_0=1}^{r_2} X''(j_0, k_0) w_{r_2}^{(j_1-1)(k_0-1)}$
(Fourier transform of each row of the grid)

It is important to emphasize that at the end of the execution of the algorithm, the resulting matrix X is transposed.

This approach to computing the FT is referred to as the fast Fourier transform (FFT) algorithm. It requires Nr_1 operations to obtain X' , and Nr_2 operations to calculate X from X'' , for a total of $N(r_1 + r_2)$ operations. We can easily see that successive applications of the above algorithm, when N can be decomposed into m factors, $N = r_1 \cdot r_2 \dots r_m$, give an m -step algorithm. If we view the N elements as being arranged into an $r_1 \times r_2 \times \dots \times r_m$, m -dimensional grid, then, in step i of the computation, elements that are adjacent over dimension $m - i + 1$ of the grid participate into FT computations. This algorithm requires a total of $N(r_1 + r_2 + \dots + r_m)$ operations. When N is a power of 2, then $r_i = 2$ for every i , and we get the classical and more well-known radix-2 FFT algorithm requiring $N \log N$ operations [5]. Another example of particular interest in this paper is when $N = 1 \cdot 2 \cdot 3 \dots n$, in which case the number of operations required is $O(Nn^2)$. For this value of N , an optimal parallel implementation would have a *processor* $\times$ *time* product of $O(Nn^2)$.

VI. THE FOURIER TRANSFORM ON THE STAR GRAPH

We are given a sequence of numbers $\{x(1), x(2), \dots, x(n!)\}$, and $n!$ processors interconnected in an S_n topology, such that each processor holds one element of the sequence. An algorithm is now presented that computes the Fourier transform of the sequence on the star graph.

In what follows, when we refer to the *rearrangement of elements over the i th connection*, we mean that all of the vertices of S_n that are adjacent through the i th connection exchange their elements.

A. Mapping the Fast Fourier Transform to the Star Graph

Assume that we want to compute the Fourier transform of a sequence of length N , where $N = n!$, on the star graph

S_n , by using the FFT method. From its definition, $n!$ can be written as follows:

$$N = n! = 1 \cdot 2 \cdot 3 \cdot 4 \cdots (n-1) \cdot n.$$

In other words, N is a number composed of n factors. If we denote the i th factor by r_i and exclude the initial unity, then $n!$ can be written as follows:

$$N = n! = r_2 \cdot r_3 \cdot r_4 \cdots r_{n-1} \cdot r_n.$$

The Fourier transform that we use is based on the decomposition of N , the number of elements, into $n-1$ factors [6]. If the N elements are arranged into an $r_2 \times r_3 \times \cdots \times r_n, (n-1)$ -dimensional grid, the $n!$ -length FFT can be computed as follows.

$r_2 \cdot r_3 \cdots r_{n-1}$	r_n -length FT's,
$r_2 \cdot r_3 \cdots r_{n-2} \cdot r_n$	r_{n-1} -length FT's,
.....	
$r_2 \cdot r_3 \cdots r_{n-i} \cdot r_{n-i+2} \cdots r_n$	r_{n-i+1} -length FT's,
.....	
$r_2 \cdot r_4 \cdots r_n$	r_3 -length FT's,
$r_3 \cdot r_4 \cdots r_n$	r_2 -length FT's.

In other words, in step i of the computation, $r_2 \cdot r_3 \cdots r_{n-i} \cdot r_{n-i+2} \cdots r_n$ FT's of length r_{n-i+1} each are computed among element that are adjacent over dimension $n-i$ of the grid.

Using the notation, and recalling the decomposition into disjoint cycles described in Section III, it follows that the star graph possesses the following attributes.

$r_2 \cdots r_{n-2}$	$(r_{n-1} \cdot r_n)$ -length cycles,
$r_2 \cdots r_{n-3} \cdot r_n$	$(r_{n-2} \cdot r_{n-1})$ -length
.....	cycles,
.....	
$r_2 \cdots r_{n-i-1} \cdot r_{n-i+2} \cdot r_n$	$(r_{n-i} \cdot r_{n-i+1})$ -length
.....	cycles,
.....	
$r_4 \cdots r_n$	$(r_2 \cdot r_3)$ -length cycles,
$r_3 \cdots r_n$	r_2 -length cycles.

We notice that S_n possesses $r_2 \cdot r_3 \cdots r_{n-i-1} \cdot r_{n-i+2} \cdots r_n$ cycles of length $r_{n-i} \cdot r_{n-i+1}$ each, or $r_2 \cdot r_3 \cdots r_{n-i} \cdot r_{n-i+2} \cdots r_n$ paths of length r_{n-i+1} each. Thus, the number and length of the paths match exactly the number and the length of the FT's required in step i of the computation.

However, in order to always have the appropriate elements in adjacent positions in the cycles of the star graph, a special initial arrangement of the elements is needed, and every Fourier transform step is separated from the next by a rearrangement of the elements. Thus, step i is followed by a rearrangement of the elements over the $(n-i+1)$ st connection of the star graph. If we imagine that the elements are arranged into a $r_2 \times r_3 \times \cdots \times r_n, (n-1)$ -dimensional grid and are indexed by following successively dimensions $1, 2, \dots, n-1$, this rearrangement is needed in step i in order to bring in adjacent vertices of the star graph groups of r_{n-i+1} elements that are adjacent over dimension $n-i$ of the grid and should participate in the same FT computation. For example, in the 2-D case, this rearrangement corresponds to a change from columns to rows. In the next two subsections, we show

how using the ordering described in Section IV and successive rearrangements, elements that participate during step i in the same r_{n-i+1} -length FT are brought into vertices of the star graph that form a linear array.

B. Embedding a Grid into the Star Graph

Assume that the $n!$ elements are arranged into a $2 \times 3 \times \cdots \times n, (n-1)$ -dimensional grid following successively dimensions $1, 2, \dots, n-1$. We now show that if the input elements are initially arranged into the vertices of the star graph according to $\prec$ (described in Section IV), and if the elements are rearranged successively over connections $2, 3, \dots, n$ of the star graph, then after rearrangement over connection i of the star graph, the i elements that are adjacent over dimension $i-1$ of the grid reside in vertices of the star graph that form a linear array.

For brevity, we use the expression *the permutation associated with element i* in this subsection to mean the permutation associated with the vertex of the star graph that contains element i . If initially $\prec$ is used, then the permutations associated with the i elements that are adjacent over dimension $i-1$ of the grid can be obtained from each other as follows: If $a_1 a_2 \cdots a_i \cdots a_n$ is the permutation associated with the last of the i elements, then the permutation associated with element $i-1$ is obtained from the permutation associated with element i after we exchange symbols in positions i and d_2 , where $d_l (2 \leq l \leq i)$ is the position of the l th largest symbol among $a_1, a_2, \dots, a_i$. In general, the permutation associated with element $i-k, 1 \leq k \leq i-1$, is obtained from the permutation associated with element $i-k+1$ by exchanging symbols in positions i and d_{k+1} . In other words, the permutation associated with element $i-k+1$ has the k th largest symbol among $a_1, a_2, \dots, a_i$ in position i . This is a property of the ordering described in Section IV.

After rearrangement of the elements in the star graph over connections $2, 3, \dots, i$, the permutation $a_1 \cdot a_2 \cdots a_i \cdots a_n$ associated with an element is transformed into $a_i \cdot a_1 \cdots a_{i-1} \cdot a_{i+1} \cdots a_n$. Now the permutation associated with element $i-k, 1 \leq k \leq i-1$ (among the i elements that are adjacent over dimension $i-1$ of the grid) is obtained from the permutation associated with element $i-k+1$ by exchanging symbols in positions 1 and d_{k+1} . However, we know from Section III that the permutations that are obtained by visiting successively connections $d_2, d_3, \dots, d_i$ of the star graph are adjacent and form the cycles that are embedded in a substar S_i of S_n . In other words, after the i th rearrangement of the elements in the star graph, elements that are adjacent over dimension $i-1$ of the grid reside in processors that form a linear array. Similar embedding of grids into the star graph can be found in [8], [13].

C. Initial Ordering and Rearrangements

Initially, we want elements that are adjacent over dimension $n-1$ of the grid to reside in processors that form a linear array. This can be done if we first rearrange elements ordered according to $\prec$ over connections $2, 3, \dots, n$ of the star graph. If an element is initially stored in the processor indexed

TABLE II
THE INITIAL ORDERING OF THE VERTICES OF THE S_4
GRAPH FOR THE FOURIER TRANSFORM ALGORITHM

4123	(1)	3124	(7)	2134	(13)	1234	(19)
4213	(2)	3214	(8)	2314	(14)	1324	(20)
4132	(3)	3142	(9)	2143	(15)	1243	(21)
4312	(4)	3412	(10)	2413	(16)	1423	(22)
4231	(5)	3241	(11)	2341	(17)	1342	(23)
4321	(6)	3421	(12)	2431	(18)	1432	(24)

TABLE III
COMPUTING THE FFT ON S_n

step	number of cycles	length of cycle
1	$(n-2)!$	$(n-1)n$
2	$(n-3)! \cdot n$	$(n-2) \cdot (n-1)$
3	$(n-4)! \cdot (n-1) \cdot n$	$(n-3) \cdot (n-2)$
$\vdots$	$\vdots$	$\vdots$
i	$(n-i-1)! \cdot (n-i+2) \cdots n$	$(n-i) \cdot (n-i+1)$
$\vdots$	$\vdots$	$\vdots$
$n-2$	$4 \cdot 5 \cdots (n-1) \cdot n$	$2 \cdot 3$
$n-1$	$3 \cdot 4 \cdots (n-1) \cdot n$	$1 \cdot 2$

S_n is viewed as sets of disjoint cycles of different lengths.

by the permutation $a_1 a_2 \cdots a_i \cdots a_n$, then after the $n-1$ rearrangements, it is stored in processor indexed by the permutation $a_n a_1 \cdots a_i \cdots a_{n-1}$. In other words, the processor indexed by the permutation $a_n a_1 \cdots a_i \cdots a_{n-1}$ must have the element whose index is $\Pi(a_1 \cdots a_i \cdots a_n)$. Therefore, the permutation representation of each vertex is cyclically shifted to the left once, before applying the mapping function Π to it. For example, for $n=4$, Table II shows the initial ordering of the vertices of S_4 (in column major order), along with the index of the element that each processor contains (see Fig. 3). Subsequently, the elements are rearranged over connections $n, n-1, \dots, 2$ during the execution of the algorithm, as illustrated in Figs. 4, 5, and 6 for S_4 . Therefore, when the algorithm terminates, each processor $a_1 a_2 \cdots a_i \cdots a_n$ contains the element indexed by $\Pi(a_1 a_2 \cdots a_i \cdots a_n)$. For $n=4$, Table I shows the final ordering of the vertices of S_4 (in column major order), along with the index of the element each processor contains (see Fig. 6).

D. Description of the Algorithm

The algorithm consists of $n-1$ steps as shown in Table III. In step i , $(n-i+1)$ -length Fourier transforms are computed among $n-i+1$ adjacent processors, on the disjoint cycles of the star graph, starting in each cycle from the vertex represented by its initial permutation. Note that in step i , $n-i$ sets of $n-i+1$ adjacent processors form a cycle of length $(n-i)(n-i+1)$.

We are now ready to describe the algorithm for computing the FT on the star graph in a more compact form. After the initial loading of the elements $\{x(1), x(2), \dots, x(n!)\}$ into the processors of the star graph following the initial arrangement of the elements, the algorithm essentially repeats three main steps $n-1$ times.

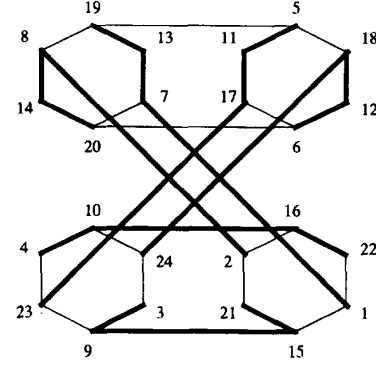


Fig. 3. Star graph FFT algorithm. Step 1: After the elements have been arranged initially as shown, this step computes 4-length FT's among the elements connected by boldface lines.

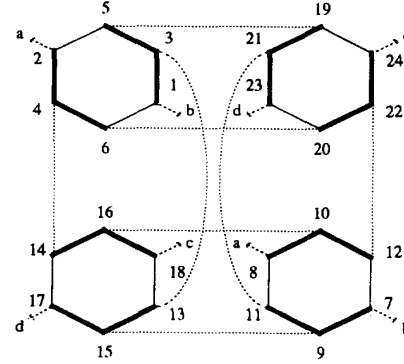


Fig. 4. Star graph FFT algorithm. Step 2: After rearrangement over the fourth connection (dotted lines) at the end of the previous step, this step computes 3-length FT's among the elements connected by boldface lines.

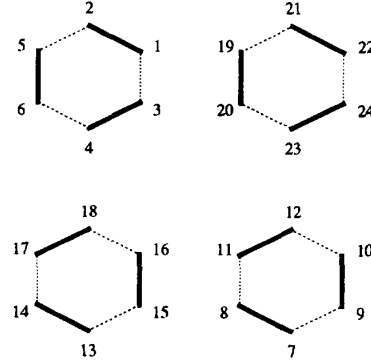


Fig. 5. Star graph FFT algorithm. Step 3: After rearrangement over the third connection (dotted lines) at the end of the previous step, this step computes 2-length FT's among the elements connected by boldface lines.

Algorithm StarFFT(n):

- 1) For $i = 1, 2, \dots, n-1$ do
 - a) Perform $(n-i+1)$ -length Fourier transforms on $n-i+1$ processors which are adjacent on the $(n-i)(n-i+1)$ -length cycles starting from the initial vertex of each cycle.
 - b) Multiply each element locally with an appropriate coefficient.

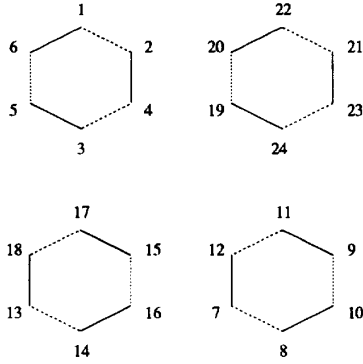


Fig. 6. Star graph FFT algorithm. Final arrangement of the elements after the rearrangement over the second connection (dotted lines) at the end of step 3.

- c) Rearrange the elements over the $(n - i + 1)$ st connection. $\square$

The elements are output following the final ordering of the processors described in Section IV. The steps of the algorithm for S_4 are illustrated in Figs. 3–6, where each vertex is labeled with the index of the element it contains.

E. Making the Algorithm Adaptive

One could argue that the $n!$ input length requirements of the algorithm might be too restrictive for an actual implementation. In what follows, we show how the algorithm can be adapted to run for an input sequence of any length on S_n . Let us assume that the Fourier transform of an input sequence of length $l \neq n!$ is to be computed on S_n . The input sequence is initially padded with zeros¹ until its length becomes $kn!$, where $k = l \text{ div } n! + 1$. One more factor, namely, k , is now added to the computation of the fast Fourier transform. A processor whose number is i according to the initial ordering in subsection VI-C now receives k elements numbered $jn! + i$, for $0 \leq j \leq k - 1$. A high-level description of the algorithm is given below.

Algorithm AdaptiveStarFFT(n, k):

- 1) Compute locally a Fourier transform of length k .
- 2) Multiply each of the k elements that a processor holds locally with an appropriate coefficient.
- 3) For $i = 1, 2, \dots, n - 1$ do
 - a) For $j = 1, 2, \dots, k$ do
 - Perform $(n - i + 1)$ -length Fourier transforms on the j th numbered elements of the $n - i + 1$ processors that are adjacent on the $(n - i)(n - i + 1)$ -length cycles starting from the initial vertex of each cycle.
 - Multiply the j th numbered element of each processor locally with an appropriate coefficient.
 - b) For $j = 1, 2, \dots, k$ do
 - Rearrange the j th numbered element of each processor over the $(n - i + 1)$ st connection.

$\square$

¹as is usually done for FT algorithms that require the length of the input sequence to be a power of 2.

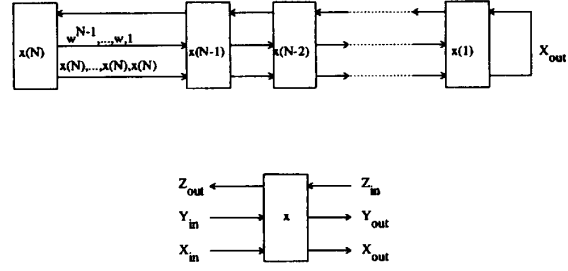


Fig. 7. The linear systolic array for computing the Fourier transform.

After the execution of the algorithm, a processor whose number is i in the final ordering holds k elements numbered $jn! + i$ for $0 \leq j \leq k - 1$. It is important to note that when $N \neq n!$, the number of zeros required to pad the sequence to reach the next $kn!$ is smaller than the number of zeros required to reach the next power of 2.

VII. THE FOURIER TRANSFORM ON THE LINEAR ARRAY

In each of the substeps of algorithms **StarFFT** and **AdaptiveStarFFT**, Fourier transforms of lengths $n, n - 1, \dots, 2$ are computed by using the linear array algorithm, which we briefly describe in this section.

The N -point Fourier transform of an input sequence $\{x(1), x(2), \dots, x(N)\}$ can be viewed as the evaluation of the following polynomial:

$$x(N)a^{N-1} + x(N-1)a^{N-2} + \dots + x(2)a + x(1),$$

at $a = 1, w, w^2, \dots, w^{N-1}$, which by Horner's rule can be written as follows:

$$(\dots((x(N)a + x(N-1))a + x(N-2))a + \dots + x(2))a + x(1).$$

This algorithm can be executed by a linear array of N multiply-add cells as shown in Fig. 7 [17]. The basic cell of this array can be seen in the same figure and its function is:

- 1) $Z_{\text{out}} = Z_{\text{in}}$,
- 2) $Y_{\text{out}} = Y_{\text{in}}$,
- 3) $X_{\text{out}} = X_{\text{in}}Y_{\text{in}} + x$.

The X_{out} that come out of the rightmost cell are the FT of the input sequence. All the X_{out} 's are computed in a pipeline fashion in $O(N)$ multiply-add steps.

This algorithm can be used for the substeps of algorithms **StarFFT** and **AdaptiveStarFFT**. In each of the substeps, the processors that must perform Fourier transforms are connected in a linear array. The only difference is that the FT's that are output by the rightmost cell must be fed back to the linear array.

VIII. CONCLUSION

Because the linear array algorithm used to compute the Fourier transform runs in $O(k)$ multiply-add steps for an input sequence of length k , algorithm **StarFFT** runs in:

$$r_n + r_{n-1} + \dots + r_3 + r_2,$$

multiply-add steps, and because for each $i, r_i \leq n$, the algorithm runs in $O(n^2)$ multiply-add steps for an input sequence of length $n!$.

As pointed out in Section V, the running time of the sequential algorithm is $N(2 + 3 + \dots + n) = O(Nn^2)$ when $N = 2 \cdot 3 \cdot \dots \cdot n$. The cost of our parallel implementation on the star graph, i.e., the *processor* $\times$ *time* product, is $O(Nn^2)$. This means that the parallel algorithm **StarFFT** is *cost-optimal* with respect to the sequential algorithm on which it is based.

For an input sequence of length $kn!$, the algorithm **AdaptiveStarFFT** includes a sequential k -length FFT computation by each processor, and each iteration of the main loop in **StarFFT** itself has been replaced by k iterations, one for each of the k elements a processor holds. Thus, the running time of this algorithm is $O(k \log k + kn^2) = O(kn^2)$, which is larger than that of algorithm **StarFFT** only by a factor of $O(k)$. As a consequence, we say that the algorithm is *optimally adaptive*.

Compared with other parallel algorithms for the Fourier transform, this algorithm is efficient because it uses the same number of processors as the number of input elements, its running time is $O(n^2)$ for an input sequence of length $n!$, and it can be made adaptive.

The best-known parallel algorithms for the fast Fourier transform are the algorithms on the hypercube [12] and the perfect shuffle [14]. Both of these networks use the same number of processors as the number of input elements. Their running time is $O(\log N)$ multiply-add steps for an input sequence of length N . When $N = n!$, these algorithms run in $O(\log n!) = O(n \log n)$ multiply-add steps.

The Fourier transform on networks like the omega network [10], the indirect binary n -cube [10], [11], the R network [10] and the butterfly network [15], need $O(n \log n)$ multiply-add steps for $n!$ elements, but use a larger number of processors, namely $O(n! \log n!)$. However, input sequences in these networks can be pipelined, leading to a constant pipeline delay after the initial $O(\log N)$ time that is needed to fill the pipeline. A large number of processors is also required for the cube-connected-cycles (CCC) [10], [12] with $O(n! \log n!)$ processors and for the mesh of trees FT [3] with $O((n!)^2)$ processors.

Algorithms like the linear array multiplication [3], the linear array FT [15], the linear systolic array [17], and the cascade FT implementation [16], which use the same number of processors as the number of input elements, involve an $O(n!)$ factor in their running time for an input sequence of length $n!$. A comparative survey of parallel algorithms for the FT is provided in [7].

ACKNOWLEDGMENT

The authors wish to thank the anonymous referees whose comments helped improve the presentation of this paper.

REFERENCES

- [1] S. B. Akers, D. Harel, and B. Krishnamurthy, "The star graph: An attractive alternative to the n -cube," *Proc. Int. Conf. Parallel Process.*, 1987, pp. 393-400.
- [2] S. B. Akers and B. Krishnamurthy, "A group-theoretic model for symmetric interconnection networks," *IEEE Trans. Comput.*, vol. 38, pp. 555-566, 1989.
- [3] S. G. Akl, *The Design and Analysis of Parallel Algorithms*. Englewood Cliffs, NJ: Prentice-Hall, 1989.

- [4] S. G. Akl, J. Duprat, and A. G. Ferreira, "Building Hamiltonian circuits and paths in star graphs," Tech. Rep. 90-22, Laboratoire de l'Informatique du Parallélisme, Ecole Normale Supérieure de Lyon, France, Mar. 1990.
- [5] W. T. Cochran, J. W. Cooley, D. L. Favon, H. D. Helms, R. A. Kaenel, W. W. Lang, G. C. Maling, D. E. Nelson, C. M. Rader, and P. D. Welch, "What is the fast Fourier transform?" *IEEE Trans. Audio Electroacoust.*, vol. AU-15, pp. 45-55, 1967.
- [6] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Math. Comput.*, vol. 19, pp. 297-301, 1965.
- [7] P. Fragooulou, "Parallel algorithms for the Fourier and other mathematical transforms," M.Sc. thesis, Dept. of Computing and Inform. Sci., Queen's University, Kingston, ON, Canada, Aug. 1990.
- [8] A. Menn and A. K. Somani, "An efficient sorting algorithm for the star graph interconnection network," *Proc. Int. Conf. Parallel Process.*, vol. III, 1990, pp. 1-8.
- [9] M. Nigam, S. Sahni, and B. Krishnamurthy, "Embedding Hamiltonians and hypercubes in star interconnection graphs," *Proc. Int. Conf. Parallel Process.*, vol. III, 1990, pp. 340-343.
- [10] D. S. Parker, "Notes on shuffle/exchange-type switching networks," *IEEE Trans. Comput.*, vol. C-29, pp. 213-223, 1980.
- [11] M. C. Pease, "The indirect binary n -cube microprocessor array," *IEEE Trans. Comput.*, vol. C-26, pp. 458-473, 1977.
- [12] F. P. Preparata and J. Vuillemin, "The cube-connected cycles," *Commun. ACM*, vol. 24, pp. 300-309, 1981.
- [13] K. Qiu, H. Meijer and S. G. Akl, "Decomposing a star graph into disjoint cycles," *Inform. Process. Lett.*, vol. 39, pp. 125-129, Mar. 1991.
- [14] H. S. Stone, "Parallel processing with the perfect shuffle," *IEEE Trans. Comput.*, vol. C-20, pp. 153-161, 1971.
- [15] C. D. Thompson, "Fourier transforms in VLSI," *IEEE Trans. Comput.*, vol. C-32, pp. 1047-1057, 1983.
- [16] E. H. Wold and A. Despain, "Pipeline and parallel-pipeline FFT processors for VLSI implementations," *IEEE Trans. Comput.*, vol. C-33, pp. 414-426, 1984.
- [17] C. Nian Zhang and D. Y. Y. Yun, "Multi-dimensional systolic networks for discrete Fourier transform," *Proc. 11th Ann. Int. Symp. Comput. Arch.*, 1984, pp. 215-222.

P. Fragooulou received the B.Sc. degree in computer science from the University of Crete, Heraklion, Crete, Greece, in 1989, and the M.Sc. degree in computer science from Queen's University, Kingston, ON, Canada, in 1990. She is currently pursuing a Ph.D. degree in computer science at Queen's University.

Her primary research interests include routing algorithms for interconnection networks, parallel algorithms, and architectures.

S. G. Akl (S'74-S'78-M'78-SM'85) received the Ph.D. degree in computer science from McGill University, Montréal, PQ, Canada, in 1978.

He is currently a Professor in the Department of Computing and Information Science at Queen's University, Kingston, ON, Canada. In 1990, he occupied the Louis Néel Chair at the Laboratoire de l'Informatique du Parallélisme of the Ecole Normale Supérieure de Lyon, France. His research interests are primarily in the area of algorithm design and analysis, in particular for problems in parallel computation.

Dr. Akl is author of *Parallel Sorting Algorithms* (Academic Press, 1985) and *The Design and Analysis of Parallel Algorithms* (Prentice-Hall, 1989), and co-author of *Parallel Computational Geometry* (Prentice-Hall, 1992). He is an editor of *Information Processing Letters* (North-Holland), *Journal of Cryptology* (Springer-Verlag), *Computational Geometry* (Elsevier), *Parallel Processing Letters* (World Scientific Publishing), and *Journal of Parallel Algorithms and Applications* (Gordon and Breach). He is a member of the IEEE Computer Society, ACM, and the European Association for Theoretical Computer Science.